TECHNICAL BRIEF · AUGUST 2026

GNSS-Denied Autonomy on a €500 Quadcopter

A companion-computer perception and planning stack for an ordinary analog FPV multirotor. Raspberry Pi 5, CPU only. No GPU, no ROS, no satellite fix.

COMPUTE	SENSOR	AUTOPILOT	LANGUAGE	VALIDATION
Raspberry Pi 5 **8 GB, CPU only**	**Intel RealSense** **D435i**	**ArduPilot** **MAVLink v2**	**C++17** **OpenCV, no ROS**	**Software-in-the-loop** **no flight yet**

01 Summary

A €500 analog FPV quadcopter already flies well. It has an attitude controller that works, a pilot who can fly it, and a video link. What it cannot do is *decide where to go* when the radio link is jammed and the satellite fix is gone.

This project adds that decision to an unmodified airframe: a Raspberry Pi 5 bolted on, a depth camera, and a serial line to the flight controller. The engineering interest is not the hardware — every part is off the shelf. It is that the whole design is derived from one measured sensor limit, and every subsequent choice can be traced back to it.

Claims below are tagged **MEASURED** a number behind it, from a run in this repository · **CODE** read from source · **ASSERTED** believed, not tested.

02 Constraints, fixed before anything was designed

These were the starting conditions, not discoveries. The architecture is what survives them.

CONSTRAINT	CONSEQUENCE IT FORCES
No satellite fix. Contested airspace takes GNSS first.	No global position anywhere in the flight path. Everything the planner uses is body-frame and short-lived.
CPU only. A Pi 5, no GPU, no accelerator.	Perception must fit in single-digit milliseconds per frame. Learned models are admissible only where a wrong answer is cheap.
The airframe is not modified.	The autopilot keeps the attitude loop. The Pi may only ask for a direction and a speed.
Attritable cost. The aircraft is expendable and the build must be reproducible.	No LiDAR, no custom hardware, nothing beyond a stock depth camera.
Analog video is the pilot's link and stays untouched.	The Pi taps the existing signal passively, adding zero latency to what the pilot sees.

03 The measurement that determines the architecture

A stereo camera does not have a range. It has a range *per question*. For the question this project asks — *"may I put a voxel of edge* `c` *here?"* — the answer holds only while the range error along the ray stays smaller than the cell. On the D435i at 0.25 m cells that limit is **3.54 m**. MEASURED

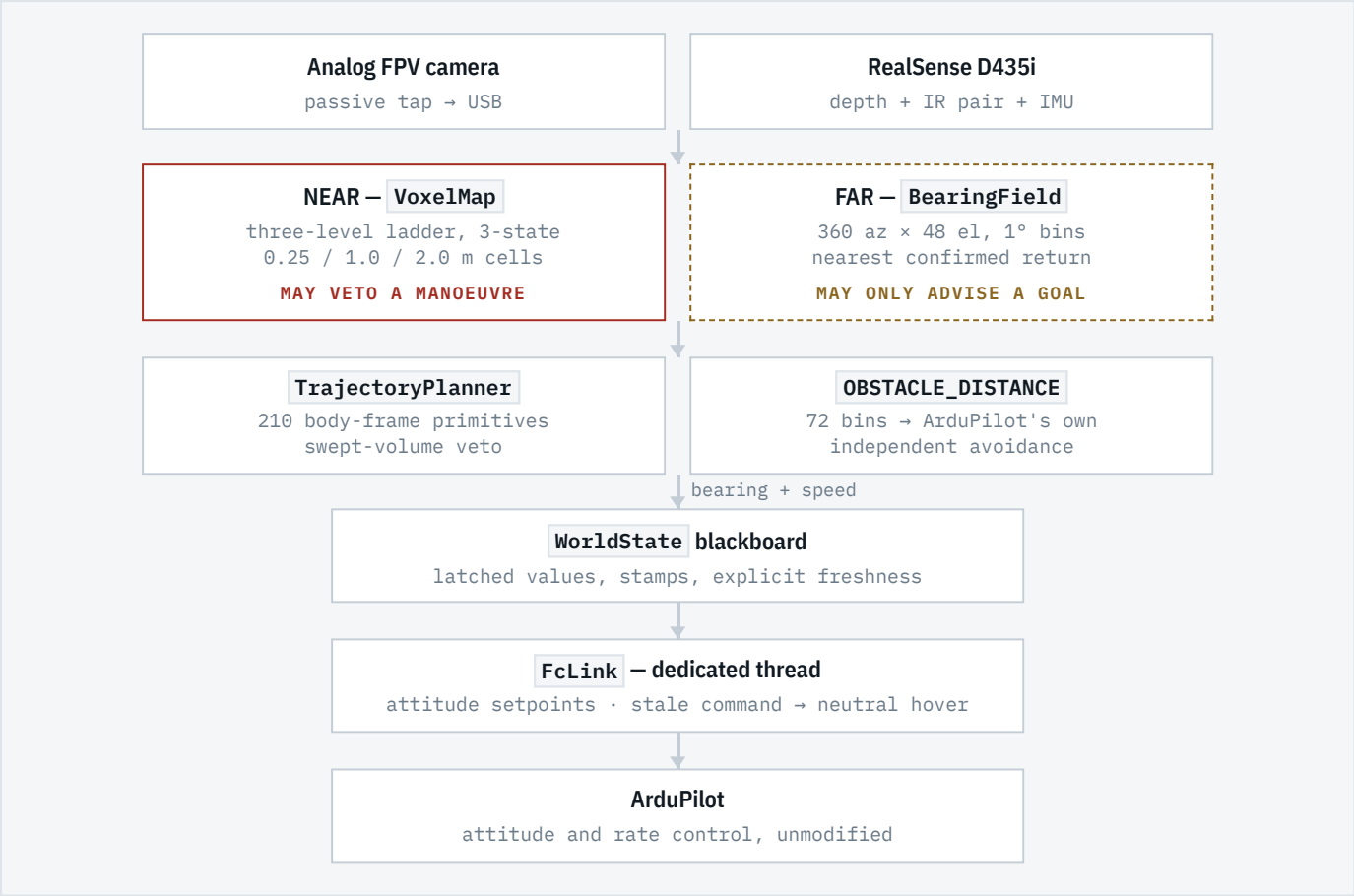
Past it the depth image still carries information, just not at voxel resolution, because stereo error is **anisotropic and increasingly so**. Along the ray it grows quadratically; across it, only linearly:

```
error ratio = Z·σ/B  →  10:1 at 2 m  ·  100:1 at 20 m
```

A cube must be sized for the worse of the two axes. A voxel honest in range at 20 m would be 8 m wide, and would discard the 4.5 cm of lateral detail the sensor still resolves there. Two representations are therefore not a convenience; they are what the error model requires.

> Cubes near. Bearings far.
>
> *Every other decision in this document is downstream of this one.*

04 Architecture



4.1 Unknown is not free

Occupancy has three states, not two, because a two-state grid cannot represent *"I have not looked there"* — so it answers *"is this tube clear?"* with *yes* for space it has never observed.

That produced a real failure, not a hypothetical one. Scoring an unobserved direction as maximally open makes **straight up** the most attractive heading in the world, because the sky returns nothing. The aircraft climbed. Unknown now scores zero, and climb and descent are penalised *symmetrically* against the commanded elevation — a one-sided penalty does not remove the bias, it inverts it, and the aircraft dives instead. MEASURED

4.2 Authority asymmetry

The two representations are deliberately not equal partners. The near field may **hard-reject** a manoeuvre whose swept 0.6 m ball touches an occupied cell; the far field contributes one weighted term to a score. So an invented near cell deletes a manoeuvre, while an invented far bearing costs only a slightly worse heading.

> Inventing awareness is cheap. Inventing permission is not.
>
> *Which is why a learned depth model is allowed to advise a direction, and never to grant one.*

4.3 Who estimates position: nobody

Three architectures were available for the link, and the choice between them is the strongest single decision in the project.

	WHO ESTIMATES POSITION	THE PI SENDS	VERDICT
A	The Pi; the autopilot consumes it	Synthetic GPS or vision pose	Retired
B	The autopilot; the Pi sends measurements	Odometry increments with covariance	Correct for v2
C	**Nobody**	Body-frame attitude commands	**Correct for v1**

Why A is not merely suboptimal but statistically wrong. ArduPilot's EKF3 treats every input as an independent measurement with a stated covariance. Feed it an estimate that has *already* been filtered on the Pi and you have two filters in series, neither aware of the other, and a covariance that no longer means anything. The failure is silent: the estimate looks plausible and is overconfident.

Why C is available at all. The thesis needs no global position — the planner is body-frame, the map is local and short-lived, and the output is a bearing and a speed. One consequence is worth stating in the design rather than discovering on the flight line: *GUIDED velocity setpoints require a horizontal velocity estimate, and without satellites there is not one.* Speed is therefore commanded as a pitch angle. Crude, and correct for the constraint.

Defence in depth, at no cost. The bearing field also publishes ArduPilot's own `OBSTACLE_DISTANCE` message, enabling a second avoidance layer *inside the autopilot* — different code, different failure modes, no dependence on our planner.

4.4 One rule, stated three times

Perception results are **latched**, so a stalled thread leaves its last result readable. That is right for control and it is a trap, because a latch from a dead thread is indistinguishable from a live one. Every result therefore carries a timestamp, consumers ask for freshness explicitly rather than trusting a valid flag, and the link thread substitutes a **neutral hover** when the planner goes stale rather than repeating a stale *motion* command.

The same rule appears in three subsystems — `unknown ≠ free` in the map, explicit freshness in the blackboard, neutral hover on the link. All three say: *absence of evidence is not evidence, and the system must be able to say so.*

05 Decision register

DECISION	CHOSEN	REJECTED, AND WHY
Flight stack	ArduPilot	Mature EKF3, documented GNSS-denied configuration, consumes our proximity output directly
Who estimates position (v1)	**Nobody**	Pi-side estimation double-filters against EKF3 and produces a meaningless covariance
Uplink interface	Attitude setpoints	Velocity setpoints need a velocity estimate that does not exist without satellites
Near representation	3-state voxels	2-state cannot express “unobserved”, the one rule the safety argument rests on
Far representation	Angular bearing bins	Stereo anisotropy makes far cubes both wasteful and dishonest
Far-field authority	Advisory only	An invented obstacle must not be able to delete a manoeuvre
Stale command	Neutral hover	Repeating a stale motion command is the worst available option
GNSS	Received, logged, routed nowhere	Makes the exclusion claim checkable by inspection instead of asserted
0.10 m near voxel rung	Retired	Cost 37% of the frame budget *and* reduced near coverage — 76.3% against 100% at 2 m standoff
Learned far-field depth	Parked	Monocular depth measured close-field-only: inverse-depth output loses resolution as $1/Z^2$

06 Evidence

Two planners, measured against each other

Six worlds × eight seeds, paired. MEASURED A swept-volume planner — *“is this entire 2 s arc clear?”* — against a VFH-style histogram planner: *“does this bearing look open?”*

The histogram planner wins progress in **all six worlds, significantly** (paired |t| from 2.76 to 17.71). Taken alone that reads as a verdict. It is not.

The swept-volume planner is not blocked, it is throttled. Instrumented, 195–207 of its 210 primitives reject on *genuinely mapped* occupied cells, with speed correctly zero. Indoors its rollout is simply longer than the room. The histogram planner never stops because it never asks for a tube — it steers. It is checking less, and that is the entirety of its advantage.

Where the extra check earns its cost it shows up as clearance, and only in the two genuinely enclosed worlds — **+0.12 m indoors** (|t| 3.41, wins 7 of 8 seeds) and **+0.61 m in a cul-de-sac** (|t| 3.69, 6 of 8). In open worlds the difference is not significant. Collisions ran 1 in 96 runs, and the swept-volume planner is **2–3× cheaper** to compute. Neither planner was retired; the measurement changed what each is *for*.

A result that reversed under sample size. At n=3 the far-field term appeared to be hurting the planner, and I said so. At n=48 the sign is the opposite: turning it off improves progress everywhere, but in the city it produced **two collisions where leaving it on produced none**. It is not a defect — it is buying safety with progress, which is the trade the whole planner makes.

> Called a bug on three samples; a safety feature on forty-eight.
>
> *The reason every claim in this document carries an evidence tag.*

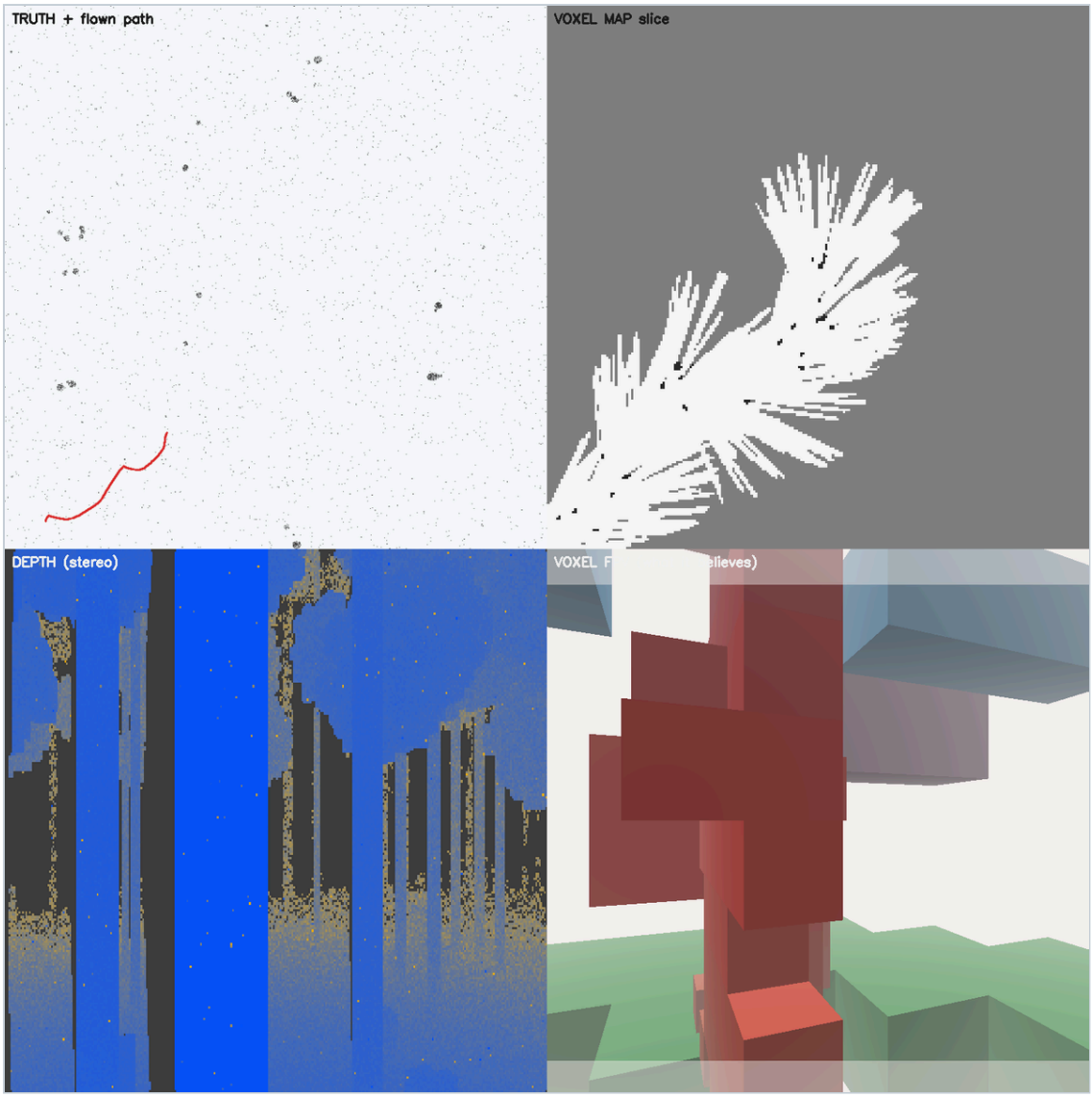
Motion estimation needs two sensors

Inertial dead reckoning alone fails, and no better IMU repairs it, because drift is dominated by attitude error and grows as t^2 while a velocity-measured error grows as t. That is a difference in order, not in quality: measured, the flight legs the planner actually commands are 3.70 s against a 1.71 s budget at one degree of attitude error. MEASURED

Optical flow scaled by per-point depth supplies the missing velocity measurement, and fusing it improves the estimate **4.1× on average and 6.9× worst-case**. The instructive part is what did *not* matter: flow *quality* barely moved the result, while flow *presence* dominated. The design question is not how accurate the flow is; it is whether there is any flow at all — which makes texture yield the variable worth measuring and accuracy the one worth ignoring.

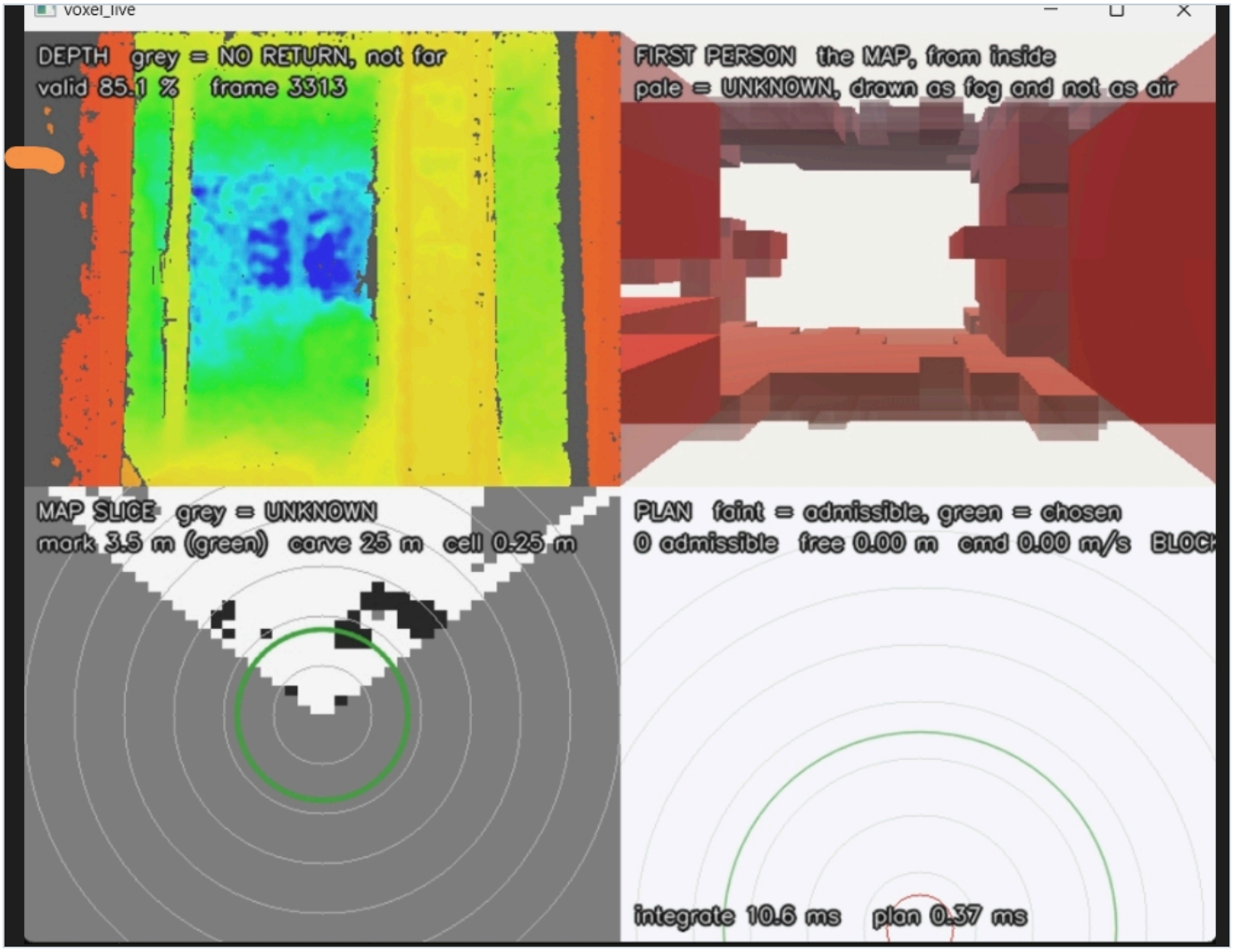
> A textureless wall yields no estimate, rather than a confident zero.
>
> *A confident zero is the lethal failure: the estimator believes it is stationary and integrates its own drift as truth.*

07 What the aircraft sees



Forest — the design case.
Top left, ground truth with the flown path, visible to the scorer and never to the planner. *Top right,* a horizontal slice of the map: white observed free, grey unknown, dark occupied. *Bottom left,* the stereo depth frame. *Bottom right,* the aircraft's own first-person render of its map — the only pane the planner can see. Colour there is height relative to the aircraft, and surfaces seen *through* unknown space are washed out in proportion to how much was traversed: a wall behind four metres of unobserved volume is a guess, and is drawn as one.

The split between the top-left and bottom-right panes is the point of the whole test harness. The aircraft plans against its own noisy map; collisions are scored against ground truth. They are different data structures on purpose — a harness that checked the plan against the map it planned with would pass no matter how wrong the map was.



The same pipeline on real sensor data, handheld indoors.
Depth 85.1% valid; integrate 10.6 ms, plan 0.37 ms. Pale is unknown, drawn as fog and not as air. The plan pane reads **0 admissible, 0.00 m/s, BLOCKED**: given a map it has not earned, the correct output is to stop, and it stops.

08 Status

Ground truth is a separate data structure from the map, so collisions are never detected against the aircraft's own belief; a perfect-depth control run accompanies every sensor run; wire formats are pinned against golden frames from an independent implementation rather than our own encoder; and any claim comparing two configurations uses multi-seed paired statistics, because single-seed results are anecdotes and one of them already reversed.

Built and measured

- Three-level 3-state voxel ladder, 6.8–9.6 ms/frame
- 360×48 bearing field, 2.5 ms
- 210-primitive planner with swept-volume veto, 2.2 ms
- Histogram planner, retained as a comparator
- MAVLink v2 codec and six decoders, golden-frame pinned
- Attitude uplink that refuses to send on a stale attitude
- Inertial odometry and a flow-to-velocity estimator
- Six simulated worlds; 25 automated checks across both trees

Not done

01 **Nothing has run on a Pi 5.** Every timing here is development-machine; a Cortex-A76 on memory-bound work is plausibly 1.5–2.5× slower. ASSERTED

02 **The depth camera has never flown.** All real captures are handheld.

03 **Flow yield on real bark and foliage is unmeasured** — and section 06 shows yield, not accuracy, is what decides the outcome.

04 **Two world models share no code**, and the older one is 2-state, so it cannot express the rule the safety argument rests on.

05 **Autopilot estimator health is decoded but not consumed.** Every safety mechanism here assumes the *map* might be wrong; none assumes the *system* might be.

06 **The non-satellite autopilot configuration is not written down.** GUIDED will refuse to engage until it is, and the symptom presents as a broken link.

Items 1–3 are what stands between this and a first flight, and none is large. Item 3 is the only one that could invalidate a design decision rather than merely delay it, which is why it is scheduled first.

Scope. A working technical demonstrator built to explore what is achievable on cheap, CPU-only hardware. Not a certified or commercially flown system; validation status is stated explicitly throughout rather than summarised favourably.

C++17 · OpenCV · MAVLink v2 · no ROS, no GPU, no CUDA.